Design and Analysis of a Multi-Store Point-of-Sale Platform with Role-Based Access, Branch-Level Inventory, Shift Reconciliation and Business Analytics: The POS System

Vrushay Bohra

Student

Department of [Department Name]

[College / Institute Name]

Vishvas Choubisa

Student

Department of [Department Name]

[College / Institute Name]

Subhash

Student

Department of [Department Name]

[College / Institute Name]

Tanya Singh

Student

Department of [Department Name]

[College / Institute Name]

Vimal Kumar

Student

Department of [Department Name]

[College / Institute Name]

Mr. Ram Babu Buri

Project Guide

Department of [Department Name]

[College / Institute Name]

Abstract—Retail businesses that operate several branches must record sales, control stock, reconcile refunds and cashier shifts, restrict staff access and understand performance, often with disconnected tools. This paper presents the design of the Point-of-Sale (POS) System, a multi-tenant web platform built on React, Spring Boot REST services, Spring Security with JSON Web Tokens, JPA/Hibernate and MySQL. Following a design-science approach, we derive a requirements baseline of 140 prioritised requirements in 26 groups, including 85 functional requirements in 12 feature groups; model the platform as a layered architecture with a store–branch tenancy hierarchy; specify security, transaction and state models; and compare the design with fifteen studies on POS, billing, multi-tenancy and access control. A consistency review of the design artefacts exposed ten discrepancies, for example client-side tax computation that the server total does not mirror, public order-retrieval routes and floating-point monetary fields, each with a proposed resolution. The source material contains no performance measurements or user study, so we make no claim of measured benefit; instead we provide a validation plan with explicit criteria for tenant isolation, transactional integrity, reconciliation accuracy and usability. The paper offers a reusable reference design and checklist for trustworthy multi-branch POS platforms.

Index Terms—Point-of-sale system, retail management, multi-tenant architecture, role-based access control, JSON Web Token, Spring Boot, inventory management, business analytics, design science.

I. INTRODUCTION

A point-of-sale (POS) system records a sale at the moment and place where it is completed. In current retail practice it is also the data source for stock control, customer records, staff accountability and management reporting. Published studies of POS adoption in small businesses report favourable operational outcomes, such as closer monitoring of overdue balances and out-of-stock items [1], and examine the factors that drive acceptance of cloud-based POS for inventory management [2]. Implementation studies describe POS introduction in settings as varied as a coffee shop [3] and a supermarket self-billing system [4].

The studies reviewed in this paper concentrate on a single enterprise or a single outlet. Many real retailers instead operate several branches under one business, and a platform operator may host many independent businesses on one deployment. This raises problems that a single-shop POS does not face: data belonging to one business must never be visible to another, stock must be tracked per branch, cashier takings must be reconciled per shift, and staff must see only what their role allows.

This paper presents the design of the POS System, a multi-tenant web platform for billing and store management, and analyses it as a design artefact. The system combines a React front end with Spring Boot REST services, Spring Security with JSON Web Tokens (JWT), Spring Data JPA/ Hibernate and a MySQL database [5], [6], [7], [8], [9]. Our contributions are:

(1) a requirements baseline of 140 prioritised requirements in 26 groups, including 85 functional requirements in 12 feature groups, written to IEEE Std 830-1998 [10] with every requirement tagged by its source;
(2) a layered architecture, a store–branch tenancy model, and security, transaction and state models for orders, refunds, shifts, stores, subscriptions and payments;
(3) a consistency review of the design artefacts that identified ten discrepancies, each with a proposed resolution; and
(4) a validation plan with measurable acceptance criteria, because the source material contains no performance measurements or user study and we make no claim of measured benefit.

Section II reviews related work, Section III describes the method, Sections IV to IX present the design, Section X reports the review findings, Section XI gives the validation plan, and Sections XII to XV discuss limitations, future work and conclusions.

II. RELATED WORK

A. POS and billing systems

Mangmang [1] developed a retail POS whose distinguishing feature is on-screen notification of overdue payments and out-of-stock items, built with Microsoft Visual Studio and SQL Server and evaluated against ISO/IEC 25010 [11] with a structured questionnaire completed by the owner and cashier. Hidayati et al. [3] analyse the implementation of a POS in a coffee shop through a case study and SWOT analysis. Natividad et al. [2] study acceptance of cloud-based POS for inventory management among micro food businesses using the Technology Acceptance Model and the IS Success model. Umar et al. [4] design and test an RFID-based supermarket self-billing system motivated by queueing time and weaknesses in existing billing processes. Together these works show the value of POS automation and the evaluation methods in use, but they address one enterprise at a time.

B. Cloud systems and multi-tenancy

Zolbanin et al. [12] examine the feasibility and challenges of cloud ERP for small and medium enterprises in a food-industry case study. Bezemer and Zaidman [13] discuss the maintenance consequences of multi-tenant software as a service, and Bezemer et al. [14] report an industrial experience of enabling multi-tenancy, contrasting the multi-tenant model, in which tenants share one application instance, with the multi-instance model, in which each tenant receives its own instance. These works motivate the single-deployment, shared-schema tenancy approach adopted here and the need to enforce isolation inside the application.

C. Access control, tokens and passwords

Sandhu et al. [15] introduced a family of reference models for role-based access control (RBAC), in which permissions are attached to roles and users are made members of roles. The NIST model [16], [17] consolidated these ideas towards a unified standard. JWT [18] defines a compact, URL-safe claims format suited to HTTP Authorization headers, and bcrypt [19] is an adaptive-cost password hash designed so that password security can keep pace with hardware improvements. These building blocks underpin the security layer of the platform.

D. Architecture and research method

Fielding [20] defines the Representational State Transfer (REST) architectural style that guides the API design. Hevner et al. [21] give guidelines for design-science research, in which knowledge is obtained by building and evaluating an artefact; we use them to frame the method. Davis [22] provides validated scales for perceived usefulness and ease of use, which we adopt for the planned user evaluation. Table I summarises the reviewed literature and its relevance.

TABLE I
SUMMARY OF REVIEWED LITERATURE AND RELEVANCE TO THIS WORK

Ref.	Focus	Method or setting	Relevance to POS
[1]	POS for retail with overdue-payment and out-of-stock notification	Visual Studio and SQL Server system; ISO/IEC 25010 questionnaire	Stock alerts; quality-model evaluation
[3]	POS implementation in a coffee shop	Case study with SWOT analysis	Adoption considerations for small retailers
[2]	Cloud POS for inventory management	TAM and IS Success model; micro food businesses	Basis for planned acceptance study
[4]	RFID self-billing for supermarkets	Document review, observation, design, implementation, testing	Alternative checkout automation
[12]	Cloud ERP for SMEs	Case study, food industry	Cloud adoption for small firms
[13]	Multi-tenant SaaS maintenance	Analysis of maintenance implications	Tenancy and maintainability trade-offs
[14]	Enabling multi-tenancy	Industrial experience report	Multi-tenant versus multi-instance
[15]	RBAC reference models	Model family	Role and permission design
[16]	NIST RBAC model	Unified model proposal	Role model consolidation
[17]	NIST RBAC standard proposal	Standard specification	Formal RBAC reference
[18]	JSON Web Token	IETF standard	Stateless authentication token
[19]	Adaptive password hashing (bcrypt)	Algorithm design	Credential storage
[20]	REST architectural style	Doctoral dissertation	API design principles
[21]	Design-science research	Methodological guidelines	Research framing
[22]	Technology acceptance scales	Scale development and validation	Planned usability and acceptance measures

Research gap. The reviewed POS studies address one enterprise or one outlet, while the multi-tenancy and access-control literature is generic. We found no reviewed work that specifies, in one place, a POS platform with a store–branch tenancy hierarchy, per-branch stock, shift reconciliation, subscription billing and role-restricted analytics, together with a consistency review of its design artefacts. This paper addresses that gap for the POS System.

III. RESEARCH METHOD

We follow the design-science approach [21], in which the artefact is the platform design and its models. The work proceeded in five steps.

Problem identification. Manual or disconnected recording of sales, stock, refunds and staff activity makes reconciliation, access control and cross-branch comparison difficult.

Requirements elicitation. The project notes [23] were organised into a software requirements specification following IEEE Std 830-1998 [10]. Each requirement has a unique identifier, a priority, a verification method and a source tag: R for behaviour documented in the project material, D for behaviour derived from it as necessary for consistency, and P for targets proposed by us and still to be confirmed. Of the 140 prioritised requirements, 97 are tagged R, 17 are tagged D and 26 are tagged P; the 26 proposed requirements are mainly quality, security, data-integrity and legal targets.

Artefact design. We produced the layered architecture, entity-relationship model, sequence models, state models and a role permission matrix.

Consistency review. The specification, sequence diagrams, state diagrams and client-routing diagrams were compared to find contradictions (Section X).

Evaluation. The design is evaluated analytically against the requirements and the reviewed literature. Empirical evaluation is specified as a validation plan (Section XI) rather than reported, because no system measurements are available.

IV. REQUIREMENTS AND SCOPE

The platform serves six user classes: SuperAdmin, Store Admin, Store Manager, Branch Admin, Branch Manager and Cashier. Its functional scope covers identity and access, store and branch administration, the product catalogue, branch-wise inventory, orders and customers, refunds, cashier shift reports, analytics, and subscriptions with online payment. Table II lists the 12 feature groups and the number of functional requirements in each.

TABLE II
FUNCTIONAL FEATURE GROUPS OF THE ZOSH POS SYSTEM

Feature group	Principal capabilities	Requirements
Authentication	Signup, login, JWT issue, profile, email-based password recovery and verification	8
Role-based access	Six roles; permission matrix; 403 on unauthorised operations	4
Store and branch	Store CRUD, PENDING/APPROVED/DECLINED moderation, onboarding, branch hours and manager	10
Product and category	Product CRUD, SKU, MRP and selling price, store-scoped keyword search	7
Inventory	Stock per product and branch, last-updated time, stock check and deduction	8
Customer	Customer records and order history	3
Order and billing	Cart, discount, customer selection, payment type, order creation, bill download	11
Refund	Refund linked to order, cashier, branch and shift; cumulative limit	7
Shift report	Start and end shift; sales, refunds, net sales, payment summary, top products	6
Analytics	Branch, store and platform dashboards; payment and category breakdowns; alerts	5
Subscription	Plans, limits, feature flags, trial, activation, renewal, cancellation	9
Payments and email	Razorpay and Stripe integration, verified confirmation, receipts and notices	7

Quality requirements are quantified as proposals to be confirmed: 95% of standard API requests within 2 s, dashboard queries within 5 s for up to 100,000 orders per store, at least 50 concurrent users with an error rate below 1%, product search within 2 s for 10,000 products, and 99% availability during store hours. Security requirements include BCrypt-hashed passwords, expiring JWTs with a configured signing secret, server-side authorisation and tenant isolation on every endpoint, parameterised queries, and no storage of card data. Barcode scanning, receipt printers, offline mode, a tax engine, PDF receipts and audit logging are explicitly deferred.

V. SYSTEM ARCHITECTURE

The platform is a layered client–server system (Fig. 1). A React single-page application, using Axios for HTTP and Redux Toolkit for state, presents role-specific interfaces: a cashier POS terminal, branch and store dashboards, a SuperAdmin console, and landing, login and onboarding pages. Requests pass through a Spring Security filter that validates the JWT and applies role and tenant checks, then reach REST controllers that exchange data transfer objects (DTOs) rather than persistence entities. Service classes, with mapper classes converting between DTOs and entities, hold the business rules; Spring Data JPA and Hibernate persist entities in MySQL, with PostgreSQL named in the notes as an alternative [23].

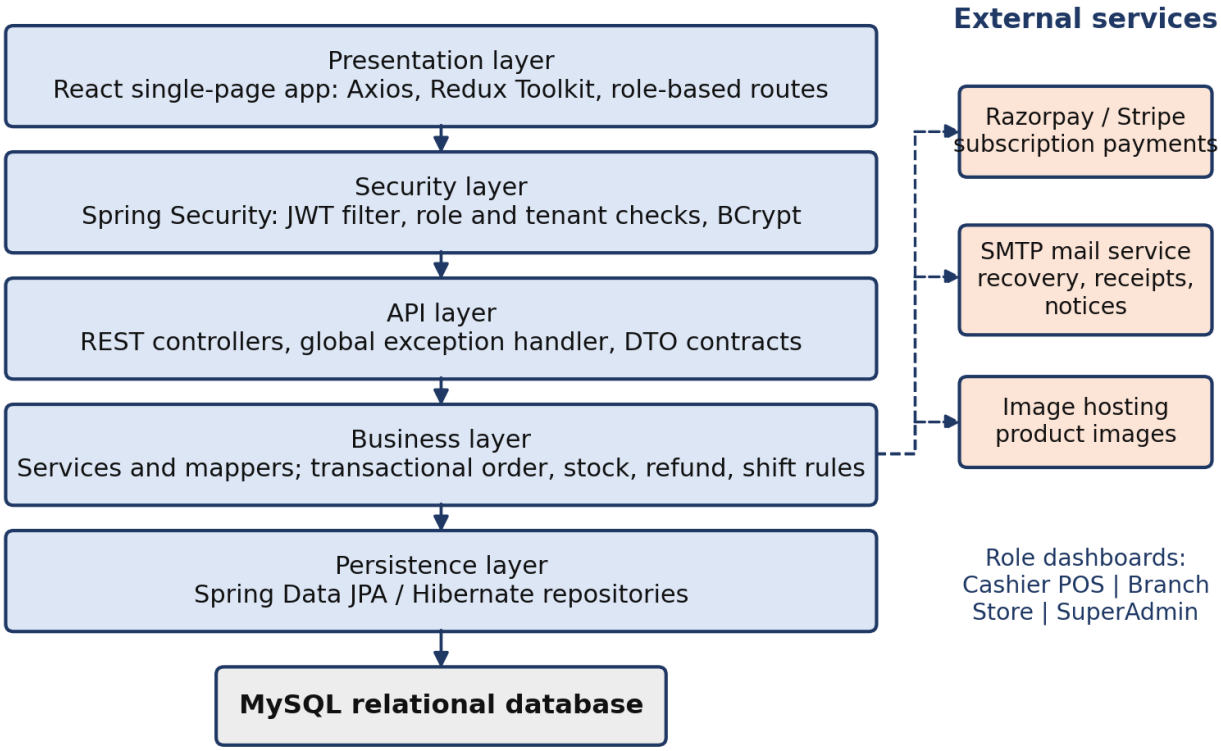


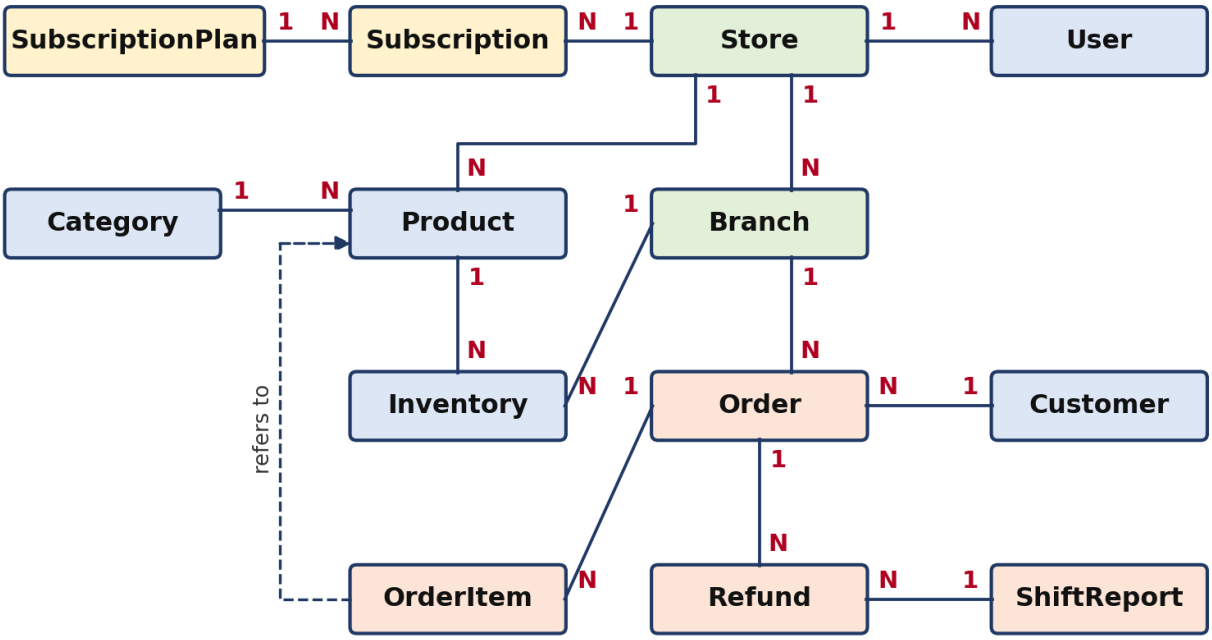
Fig. 1. Layered architecture of the POS System and its external services.

The API follows REST conventions [20]: resources such as products, stores, branches, inventories, customers, orders, refunds and shift reports are addressed by URL and manipulated with POST, GET, PUT or PATCH, and DELETE. Controllers are organised by business area, each with its own service and mapper, which keeps modules independently testable and maintainable.

Tenancy model. One deployment serves many stores. Each business record is owned by a store and, where applicable, by a branch within it, and isolation is enforced on the server for every read and write. This is the multi-tenant model discussed by Bezemer et al. [14], chosen over one instance per tenant because it reduces operational effort, at the price that the application itself must guarantee isolation [13]. The notes do not describe a deployment topology or measured capacity.

VI. DATA MODEL

The relational model is organised around stores and branches (Fig. 2, Table III). A store owns branches, users, categories, products and a subscription. A branch holds contact details, working days and hours, and an assigned manager. Inventory links a product to a branch with a quantity. An order references its branch, cashier and customer and contains order items, each recording a product, quantity and unit price. A refund references its order, cashier, branch and shift report. A shift report aggregates a cashier's sales, refunds and payment summary for one shift. A subscription links a store to a plan and records dates, status, gateway, transaction identifier and payment status.



Users with cashier role are referenced by Order, Refund and ShiftReport; Branch is also referenced by ShiftReport. Category and Product belong to one Store; User and Branch belong to one Store.

Fig. 2. Entity-relationship model of the core entities (1 = one, N = many).

TABLE III
CORE ENTITIES AND KEY ATTRIBUTES

Entity	Key attributes
User	Name, email, password hash, role, status; store or branch
Store	Brand, administrator, type, status, contact
Branch	Address, contact, working days and hours, manager
Product	SKU, name, MRP, selling price, category, store
Inventory	Product, branch, quantity, last updated
Order	Total, branch, cashier, customer, payment type, time
OrderItem	Order, product, quantity, unit price
Refund	Order, amount, reason, cashier, shift, payment type
ShiftReport	Cashier, interval, sales, refunds, net sales, counts
Subscription	Store, plan, dates, status, gateway, transaction, payment status

The design requires foreign-key integrity, a unique email per user and a unique SKU per store, fixed-point decimal types for money, unit prices stored on order items so that later price changes do not alter past orders, and indexes on the columns used for store, branch, cashier, customer and date filters. The notes provide no complete schema diagram, indexes, constraints or migration strategy, so these are design requirements rather than documented facts.

VII. SECURITY DESIGN

Authentication. A successful login returns a signed JWT [18] carrying the user's identity and role; clients send it in the Authorization: Bearer header, and the server keeps no session state. Missing, malformed or expired tokens are rejected with 401. The signing secret is supplied by configuration and never committed to source control.

Credentials. Passwords are stored only as BCrypt hashes [19]. The adaptive cost of bcrypt allows the work factor to be raised as hardware improves, and password hashes are never returned by any API.

Authorisation. Access follows the role-based model [15], [16], [17]: permissions are attached to roles and users are assigned roles (Table IV). Enforcement is on the server; hiding a button in the interface is not a control. Beyond role checks, object-level authorisation verifies that the target store or branch belongs to the caller, so changing an identifier in a request cannot expose another tenant's products, orders, employees or analytics. An unauthorised operation returns 403 and changes nothing.

TABLE IV
USER CLASSES AND PRINCIPAL PRIVILEGES

Role	Scope and principal privileges
SuperAdmin	All stores: moderate stores, manage plans, platform dashboards
Store Admin	Own store: details, branches, employees, products, subscription
Store Manager	Own store: employees, products, inventory
Branch Admin	Own branch: employees and operations
Branch Manager	Own branch: orders, employees, inventory, analytics (read)
Cashier	Own branch: create orders, customers, permitted refunds, own shifts

Payments. Card data is handled only by the gateways and is never stored. A subscription's payment status changes only after verified confirmation from the gateway, gateway callbacks have their signatures verified, and a repeated confirmation for the same transaction must not create a duplicate subscription. Security requirements are checked against the OWASP ASVS checklist [24].

VIII. CORE TRANSACTION WORKFLOWS

A. Sale

A cashier signs in, searches products within the store, builds a cart, optionally applies a discount and selects a customer, and submits the order with a payment type, which in the project material is cash, card or UPI. The backend (Fig. 3) verifies authorisation and branch scope, checks stock for every item, computes the total as the sum of unit price times quantity minus discount, and saves the order and its items and decreases stock in one database transaction. If any item lacks stock the order is rejected and nothing is saved, so there is no partial order and no overselling of the last unit. The response is the order, after which the client clears the cart and offers the bill for download.

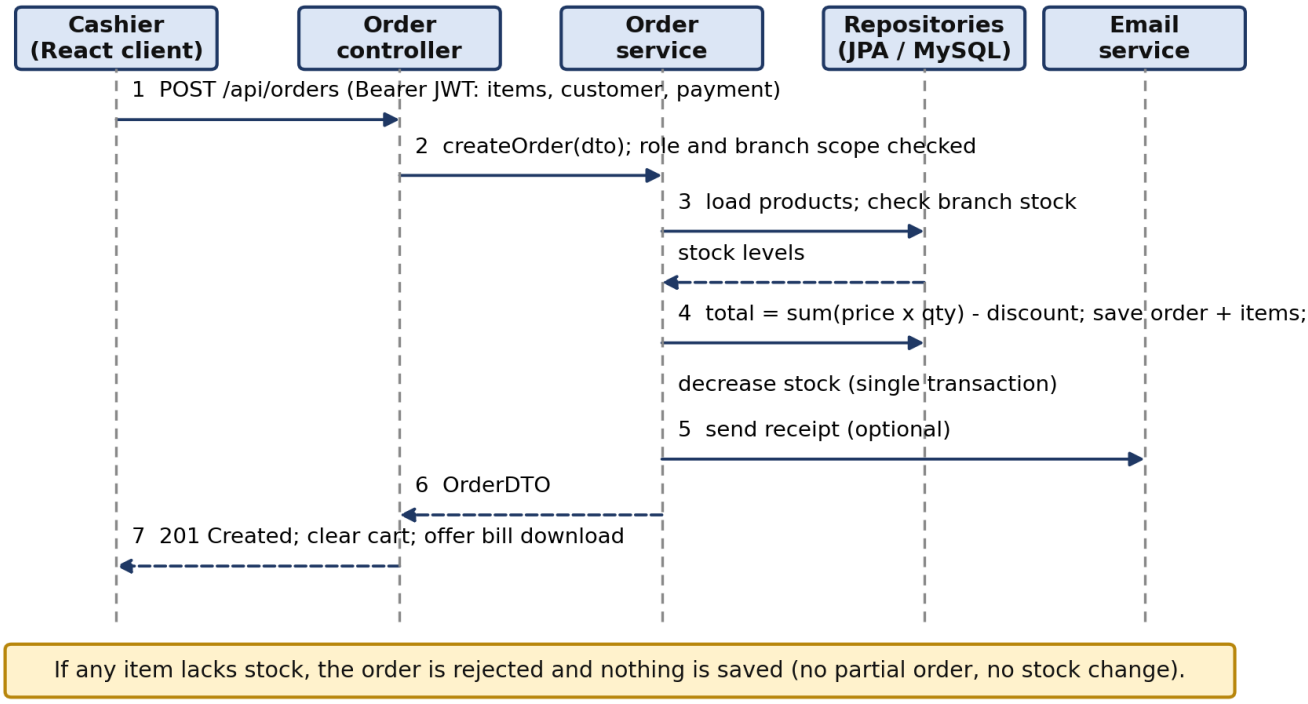


Fig. 3. Sequence of order creation: authorisation, stock check, total, atomic save and receipt.

B. Refunds and shift reconciliation

A cashier starts a shift explicitly and ends it explicitly; at most one shift per cashier may be open. Orders and refunds made meanwhile are attributed to the open shift. A refund is created against an existing order, records amount and reason, links to the order, cashier, branch and shift, and is rejected if cumulative refunds on the order would exceed the order total. Creating a refund updates the shift's refund total and net sales. When the shift ends, the system computes total sales, total refunds, net sales, order count, payment-type summaries and top-selling products from the orders and refunds recorded between start and end, so that net sales equal total sales minus total refunds.

C. Store moderation and subscription lifecycle

New stores start as PENDING; only the SuperAdmin may approve or decline them, and the store administrator is notified by email (Fig. 4). A subscription starts as TRIAL, becomes ACTIVE when payment is confirmed as PAID, becomes EXPIRED when the trial or the end date passes, may be renewed by payment, and ends as CANCELLED when an administrator cancels it. Payment status moves from PENDING to PAID or FAILED, and a FAILED payment may be retried. Explicit state models make illegal transitions testable: any transition not in the model is rejected.

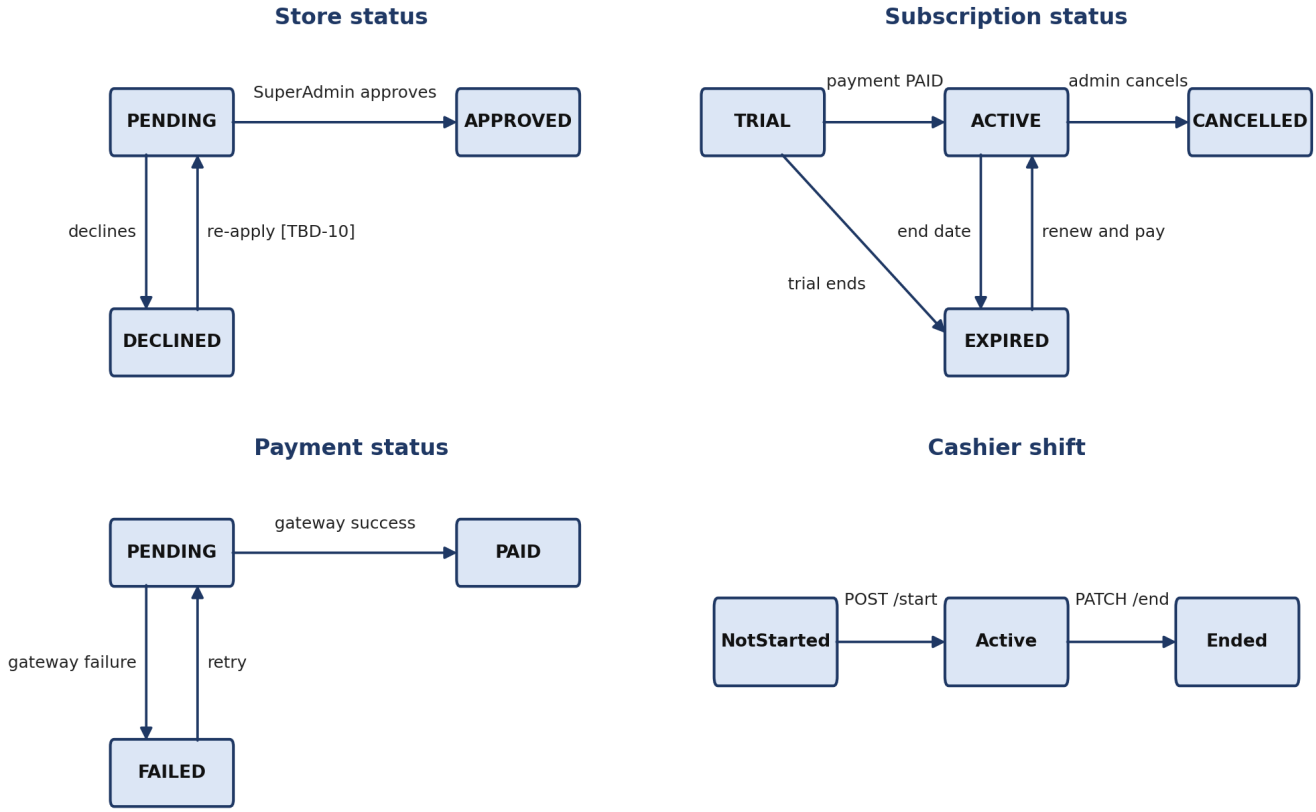


Fig. 4. State models of store status, subscription status, payment status and cashier shift.

IX. ANALYTICS AND REPORTING

The analytics endpoints provide branch views (daily sales, top-selling products, top cashiers by revenue, category-wise sales, today's overview and payment-method breakdown), store views (key indicators, sales trends, monthly and daily sales, branch comparison, category and payment distribution, and operational alerts such as low stock) and platform views for the SuperAdmin (store summary, registration trend and status distribution). Every view is scoped to the caller's store or branch.

Reporting needs fixed definitions. Net sales are defined as total sales minus total refunds. The implementation must also decide whether totals are gross or net of discounts and taxes, how cancelled orders are treated, which timestamp determines the reporting day, and whether a refund is attributed to the original sale date or to the refund date. The design requires that analytics be computed from the same persisted orders and refunds used for billing, so dashboard totals reconcile with shift reports.

X. DESIGN REVIEW FINDINGS

Comparing the specification, the sequence and state diagrams, the client-routing diagram and the project notes revealed ten discrepancies (Table V). Reporting them is a result of the design-science process [21]: each is an inconsistency that would otherwise surface as a defect during implementation or testing.

TABLE V
DISCREPANCIES FOUND IN THE DESIGN ARTEFACTS AND PROPOSED RESOLUTIONS

#	Finding	Artefacts compared	Proposed resolution
1	The checkout flow computes subtotal + 18% tax − discount on the client; the order service total is subtotal − discount; tax logic is listed as an enhancement.	Checkout sequence; order sequence; notes	Make the server the single source of truth for totals; add tax on the server or remove it from the client.
2	Some order-retrieval routes are labelled public while the security design describes secured, role-based routes.	Notes: endpoint table and security section	Document the intended access policy and secure order, customer and cashier data.
3	Some monetary fields use floating-point Double.	Notes: data model	Use fixed-point decimal types for all money.
4	Insufficient stock returns 400 in the order sequence but a business-rule status (422) in the error contract.	Order sequence; error contract	Align the implementation and the contract on one status.
5	Role identifiers differ: ROLE_SUPER_ADMIN and ROLE_CASHIER in the hierarchy diagram, ROLE_ADMIN and ROLE_BRANCH_CASHIER in client routing.	Role hierarchy; routing diagram	Define one identifier set shared by back end and client.
6	A new subscription starts as TRIAL in the state diagram, TRIAL or PENDING in the payment sequence, and PENDING_PAYMENT in an earlier draft.	State diagram; payment sequence; draft SRS	Adopt TRIAL and update all artefacts.
7	The state diagram allows a DECLINED store to re-apply; the policy was undecided.	Store state diagram; draft SRS	Confirm and document the re-application policy.
8	The order sequence decreases stock, but the refund sequence does not restore it.	Order and refund sequences	Decide whether refunds restore stock, in the same transaction.
9	Gateway SDKs and payment fields are listed, but webhook verification, reconciliation and failure handling are not demonstrated.	Notes: payment and subscription sections	Specify and test callbacks, idempotency and failure paths before claiming support.
10	The state diagram says an administrator cancels a subscription; an earlier draft says the store cancels.	State diagram; draft SRS	Define whether the Store Admin, SuperAdmin or both may cancel.

XI. VALIDATION PLAN

The source material provides no test results, so this section specifies how the design should be validated. The listed tools in the notes, namely Spring Boot Test, JUnit, Mockito, Spring Security Test and Postman, support the plan. Table VI gives the evidence required at each level. Quality is assessed against ISO/IEC 25010 [11], as in [1], and user acceptance with the perceived-usefulness and ease-of-use scales of Davis [22], as in [2]. The numeric targets are the proposed requirements of Section IV and must be confirmed before use. All measurements must state the environment, data set, concurrency and method, and must be collected rather than estimated.

TABLE VI
VALIDATION PLAN AND ACCEPTANCE CRITERIA

Level	What is verified	Acceptance criterion (proposed)
Unit	Totals, discounts, stock rules, refund limits, shift arithmetic, subscription and payment transitions, role checks	All rule tests pass; 70% line coverage of the service layer
Integration	Entity relationships, database constraints, transactional order creation	No partial order and no stock change after an injected failure
API	Valid and invalid requests, 401, 403, 404, 400 and validation errors	Error responses follow one documented contract
Tenant isolation	A user of one store reading or changing another store or branch by ID	100% of cross-tenant attempts denied for every endpoint
Concurrency	Two cashiers selling the last unit of a product	Exactly one sale succeeds; stock never negative
Reporting	Dashboard aggregates against a controlled set of orders and refunds	Zero difference from expected totals and from shift reports
Performance	Response-time percentiles under a stated multi-branch workload	95% of standard requests within 2 s; analytics within 5 s at 100,000 orders
Usability	Cashier task completion; perceived usefulness and ease of use	Basic sale in at most 6 interactions; acceptance scores reported with sample size
Operations	Backup and restore, logging, recovery from database and gateway failure	Documented recovery procedure exercised successfully

XII. DISCUSSION

The main strength of the design is functional breadth: checkout connects to inventory, staff, customers, refunds, shifts and analytics, and the store and branch entities provide a path to multi-location management. The controller–service–repository pattern separates HTTP concerns from business rules and persistence, and DTOs reduce direct exposure of database entities. JWT and role-based authorisation are mature building blocks for a web client consuming a REST API [18], [15].

The main risks are cross-store data leakage, inconsistent stock during concurrent sales, unclear public access to business records, monetary precision, and ambiguity between documented and implemented behaviour. The design-review findings show that the first three are not hypothetical: they appear as concrete inconsistencies in the artefacts. The discipline of tagging each requirement as documented, derived or proposed keeps the boundary between established behaviour and design guidance visible, which is particularly important for payment processing, where SDKs are listed but completed end-to-end flows are not demonstrated.

Threats to validity. The analysis rests on project documentation rather than a source-code audit, so the described behaviour is documented or proposed, not verified. The consistency review compares artefacts that may themselves be incomplete. The literature reviewed is limited to the studies cited and is not a systematic review. The proposed numeric targets are not derived from measurement.

XIII. LIMITATIONS

This paper is based on 26 pages of project notes, a requirements specification, and the project's design diagrams. These are feature and design documentation, not a complete code audit or empirical study. The material provides no deployment evidence, production logs, complete database schema, completed security test results, performance benchmarks, user feedback or verified payment-gateway transactions. Some endpoint access labels and model descriptions need implementation-level confirmation. Recommendations are therefore design guidance and not established results, and no numerical claim about sales improvement, speed, accuracy or productivity is made.

XIV. FUTURE WORK

Priorities are: (1) validating tenant isolation and role permissions with automated tests; (2) making order, stock and refund changes transactional and resolving the stock-restoration policy; (3) adding schema migrations, indexes, backup and recovery procedures; (4) specifying and testing discounts, taxes, receipt generation and payment reconciliation; (5) integrating barcode scanners where required; (6) exploring offline operation with conflict resolution; (7) measuring performance under realistic multi-branch workloads; and (8) evaluating usability and acceptance with cashiers and managers using established scales [22]. Payment-gateway integration should be described as implemented only after end-to-end payment and failure-path tests confirm it.

XV. CONCLUSION

The POS System is designed as a multi-store retail management platform that combines point-of-sale operations with product and inventory control, employee and customer management, refunds, cashier shifts, analytics and subscription administration. React, Spring Boot REST services, JPA/Hibernate, MySQL, Spring Security and JWT form its technical foundation. We contributed a 140-requirement baseline, a layered and multi-tenant architecture, security, transaction and state models, a review that exposed ten design discrepancies, and a validation plan with measurable criteria. Dependable deployment requires more than a broad endpoint list: tenant-level authorisation, transactional stock updates, precise monetary handling, consistent access rules and systematic testing are essential. Because no empirical evaluation was available, the contribution is a structured design analysis and validation plan, not a claim of measured business impact.

ACKNOWLEDGMENT

The authors thank their project guide, Mr. Ram Babu Buri, for guidance and review throughout the development of the POS System and this paper.

REFERENCES

[1] G. Mangmang, “Development and implementation of point of sale system (POS): Profitability measurement for retail business,” SLSU J. Educ. Human Resour. Develop. (JEHRD), vol. 6, pp. 220–226, 2018, doi: 10.61569/5tc08q53.

[2] E. A. E. Natividad et al., “System quality and micro food businesses’ acceptance of the cloud-based point of sale system for inventory management,” in Proc. 15th Int. Conf. E-business, Manage. Econ. (ICEME), 2024, pp. 123–129, doi: 10.1145/3691422.3691480.

[3] I. Hidayati, S. Sudarmiatin, and A. Hermawan, “Analysis of point of sale system implementation in coffee shop,” Jurnal Pendidikan dan Kewirausahaan, vol. 11, no. 3, pp. 932–947, 2023, doi: 10.47668/pkwu.v11i3.881.

[4] S. Umar, S. Twaibu, G. G. Ocen, L. Badru, A. Andrew, and M. Davis, “RFID-based automated supermarket self-billing system,” East Afr. J. Inf. Technol., vol. 4, no. 1, pp. 7–15, 2021, doi: 10.37284/eajit.4.1.488.

[5] Spring, “Spring Boot reference documentation.” [Online]. Available: https://docs.spring.io/spring-boot/

[6] Spring, “Spring Security reference documentation.” [Online]. Available: https://docs.spring.io/spring-security/reference/

[7] Spring, “Spring Data JPA reference documentation.” [Online]. Available: https://docs.spring.io/spring-data/jpa/reference/

[8] React, “React documentation.” [Online]. Available: https://react.dev/

[9] MySQL, “MySQL 8.4 reference manual.” [Online]. Available: https://dev.mysql.com/doc/refman/8.4/en/

[10] IEEE Recommended Practice for Software Requirements Specifications, IEEE Std 830-1998, 1998.

[11] Systems and Software Engineering: Systems and Software Quality Requirements and Evaluation (SQuaRE): System and Software Quality Models, ISO/IEC 25010:2011, 2011.

[12] H. M. Zolbanin, A. H. Zadeh, B. A. Akinyemi, and A. Jeyaraj, “Cloud ERP systems for small-and-medium enterprises: A case study in the food industry,” J. Cases Inf. Technol., vol. 20, no. 4, pp. 53–70, Oct. 2018, doi: 10.4018/jcit.2018100104.

[13] C.-P. Bezemer and A. Zaidman, “Multi-tenant SaaS applications: Maintenance dream or nightmare?,” in Proc. Joint ERCIM Workshop Softw. Evol. (EVOL) and Int. Workshop Principles Softw. Evol. (IWPSE), New York, NY, USA, 2010, pp. 88–92.

[14] C.-P. Bezemer, A. Zaidman, B. Platzbeecker, A. A. M. Hurkmans, and A. ’t Hart, “Enabling multi-tenancy: An industrial experience report,” in Proc. 26th IEEE Int. Conf. Softw. Maintenance (ICSM), 2010, pp. 1–8.

[15] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, “Role-based access control models,” Computer, vol. 29, no. 2, pp. 38–47, Feb. 1996.

[16] R. Sandhu, D. Ferraiolo, and R. Kuhn, “The NIST model for role-based access control: Towards a unified standard,” in Proc. 5th ACM Workshop Role-Based Access Control, Berlin, Germany, Jul. 2000, pp. 47–63.

[17] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, “Proposed NIST standard for role-based access control,” ACM Trans. Inf. Syst. Secur., vol. 4, no. 3, pp. 224–274, Aug. 2001.

[18] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” IETF RFC 7519, May 2015.

[19] N. Provos and D. Mazières, “A future-adaptable password scheme,” in Proc. USENIX Annu. Tech. Conf., Monterey, CA, USA, Jun. 1999, pp. 81–92.

[20] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, USA, 2000.

[21] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” MIS Q., vol. 28, no. 1, pp. 75–105, Mar. 2004.

[22] F. D. Davis, “Perceived usefulness, perceived ease of use, and user acceptance of information technology,” MIS Q., vol. 13, no. 3, pp. 319–340, Sep. 1989, doi: 10.2307/249008.

[23] POS System, “Project notes,” 26-page technical documentation supplied by the project authors, undated, unpublished.

[24] OWASP Foundation, “OWASP Application Security Verification Standard (ASVS),” version 4.0.